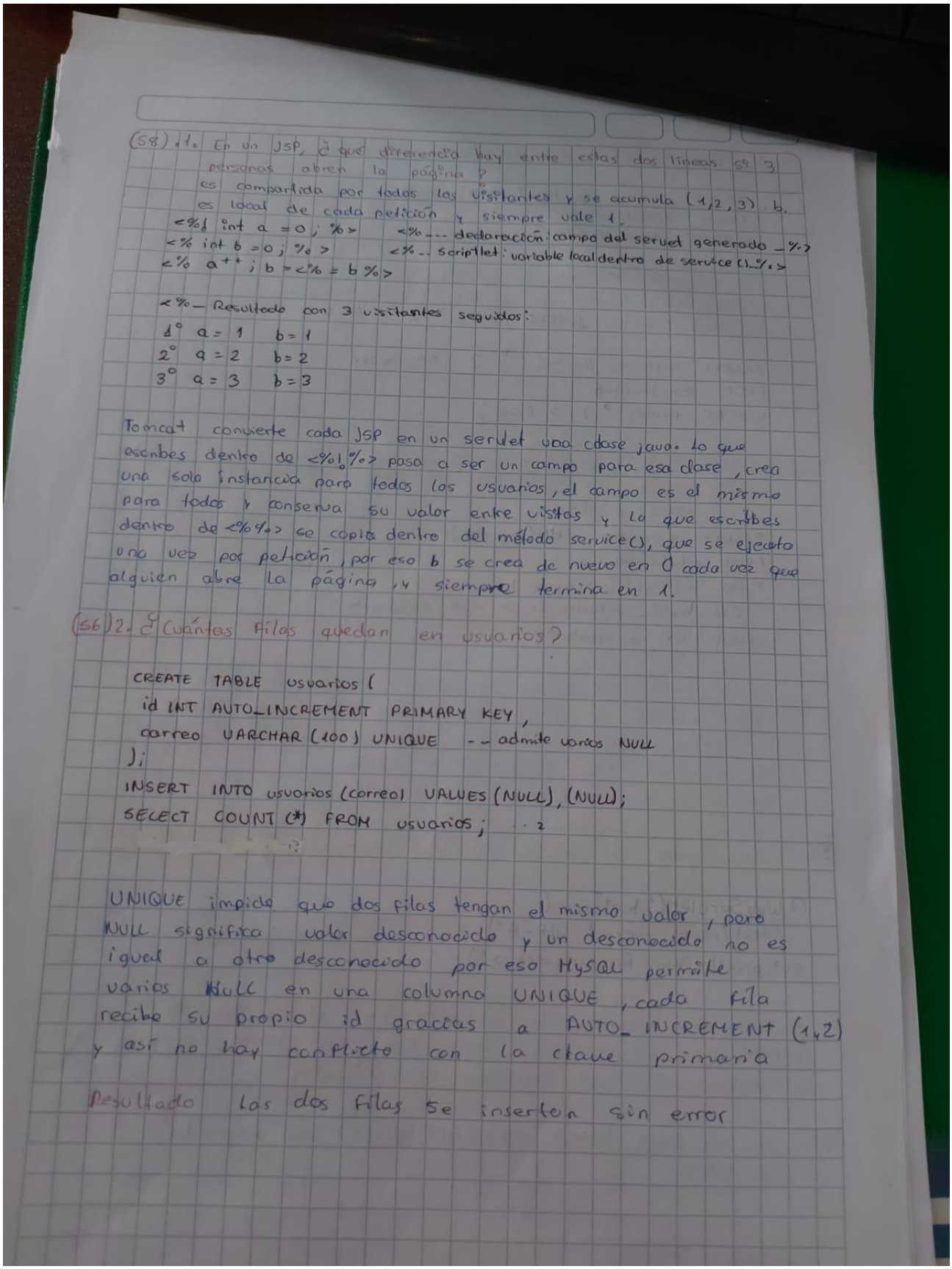
Resolución de las 80 preguntas del examen

Nombre : Margori Lizet Cayllahua Vargas



(58) 1. En un JSP, ¿qué diferencia hay entre estas dos líneas si 3 personas abren la página?

a. es compartida por todas las visitantes y se acumula (1, 2, 3)
b. es local de cada petición y siempre vale 1.

```
<%! int a = 0; %>        <%-- declaración: campo del servlet generado --%>
<% int b = 0; %>         <%-- scriptlet: variable local dentro de service() --%>
<% a++; b = <% = b %>
```

<%-- Resultado con 3 visitantes seguidos:

1° a = 1 b = 1
2° a = 2 b = 2
3° a = 3 b = 3

Tomcat convierte cada JSP en un servlet una clase java. Lo que escribes dentro de <%! %> pasa a ser un campo para esa clase, crea una sola instancia para todos los usuarios, el campo es el mismo para todos y conserva su valor entre visitas y lo que escribes dentro de <% %> se copia dentro del método service(), que se ejecuta una vez por petición, por eso b se crea de nuevo en 0 cada vez que alguien abre la página, y siempre termina en 1.

(56) 2. ¿Cuántas filas quedan en usuarios?

```
CREATE TABLE usuarios (
 id INT AUTO_INCREMENT PRIMARY KEY,
 correo VARCHAR(100) UNIQUE   -- admite varios NULL
);
INSERT INTO usuarios (correo) VALUES (NULL), (NULL);
SELECT COUNT(*) FROM usuarios;   -- 2
```

UNIQUE impide que dos filas tengan el mismo valor, pero NULL significa valor desconocido y un desconocido no es igual a otro desconocido por eso MySQL permite varios NULL en una columna UNIQUE, cada fila recibe su propio id gracias a AUTO_INCREMENT (1, 2) y así no hay conflicto con la clave primaria

Resultado las dos filas se insertan sin error

3. ¿Qué ocurre al ejecutar esta consulta?

```
SELECT nombre, puntos * 2 AS doble
FROM estudiantes
WHERE doble > 600;
```

La consulta genera un error porque no se puede utilizar alias doble en el WHERE de esa misma consulta, se debe escribir WHERE puntos * 2 > 600;

```
SELECT nombre, puntos * 2 AS doble
FROM estudiantes
WHERE puntos * 2 > 600;
```

4 Carlos (500 puntos) tiene 2 canjes. ¿Qué suma muestra para Carlos?

1000 puntos

```
SELECT e.nombre, SUM(e.puntos)
FROM estudiantes e
JOIN historial_canjes h
  ON h.id_estudiante = e.id_estudiante
GROUP BY e.nombre;
```

Carlos tiene 500 puntos y aparece en dos registros de canjes, Como el JOIN genera dos filas para Carlos y SUM(e.puntos) suma sus puntos en ambas y el resultado es 500 + 500 = 1000
eso es lo que muestra esta consulta

5. ¿Qué ve el usuario en el navegador?

```
response.setContentType(
  "text/plaint; charset=UTF-8");
out.println("<h1>Ranking</h1>");
```

El navegador muestra la respuesta según el encabezado ContentType, con text/plaint trata todo como texto simple y lo muestra tal cual incluidas las etiquetas

Rpta: ve el texto literal <h1>Ranking</h1>, sin formato de título

(58) 6. Se abre /juegos/5 ¿Qué devuelve request.getPathInfo()?

"/5"

```
@WebServlet("/juegos/*")
...
String p = request.getPathInfo();
```

El patrón "/juegos/*" significa el servlet atiende /juegos y todo lo que venga después, la URL se divide en partes, como servletPath = "/juegos" y pathInfo = "/5".
getPathInfo() devuelve la parte sobrante con su barra inicial, por eso es /5 y no 5, para obtener el número hay que quitar el primer caracter con substring(1).

(57) 7. ¿Qué ocurre con este alias?

```
rs = conn.prepareStatement(
    "SELECT e.nombre AS estudiante"
  + "FROM estudiante e").executeQuery();
rs.next();
rs.getString("nombre");
```

corrección

```
rs = conn.prepareStatement(
    "SELECT e.nombre AS estudiante
     FROM estudiante e"
).executeQuery();
rs.next();
rs.getString("estudiante");
```

Se produce un error SQLException; columna nombre no encontrada, el alias AS renombro la columna a estudiante y getString busca por el nombre que trae el resultado.

(57) 8. Luis tenía 200 puntos. ¿Cuantos puntos tiene después en la base de datos?

200

```
Try (Connection c = ConexionBD
    .obtenerConexion()) {
  c.setAutoCommit(false);
  c.prepareStatement("UPDATE estudiantes"
    + "SET puntos = 999"
    + "WHERE id_estudiante = 3")
  .executeUpdate();
} // nunca se llamo commit()
```

Con autocommit desactivado, cada cambio queda pendiente dentro de una transacción, solo se guarda definitiva cuando llamas commit(), en el código nadie llamo commit() al cerrarse la conexión con la transacción abierta MySQL hace el rollback y el UPDATE se descarta así que Luis sigue ~~teniendo~~ con 200.

(58) 9. Para "ahorrar código", un servlet que elimina registros hace esto ¿Qué riesgo introduce?

```
protected void doGet(...) {
  doPost(request, response);
}
protected void doPost(...) {
  dao.eliminar(id);
}
```

Que se eliminen registros accidentalmente ya que doGet() llama a doPost() y permite borrar datos al abrir un enlace, además los navegadores pueden precargar enlaces y facilitar ataques CSRF, por eso GET debe usarse solo para consultar información y las eliminaciones deben realizarse mediante POST, verificando la autenticación del usuario

(66) 10 ¿Qué nombres devuelve?

```
--- estudiantes: (1, 'Carlos Mendoza', 500),
--   (2, 'Ana Torres', NULL), (3, 'Luisa Rojas', 200)
SELECT nombre FROM estudiantes
WHERE nombre LIKE '_a%';
```

Carlos Mendoza, el comodín _ ocupa una posición y % ocupa cualquier cantidad y el "_a%" significa una letra cualquiera, luego una a y despues lo que sea, nombres cuya segunda letra es "a", Ana tiene n y Luis tiene u.

(57) 11 Carlos YA tiene 500 puntos. ¿Que devuelve executeUpdate() con Connector/J (configuración por defecto)?

```
PreparedStatement ps = conn.prepareStatement(
"UPDATE estudiantes SET puntos = 500"
+ "WHERE id_estudiante = 1");
int n = ps.executeUpdate();
```

executeUpdate() devuelve un numero de filas, puede contarlas de dos maneras, las encontradas (que cumplen el WHERE) o las afectadas (las que realmente cambiaron).
Connector/J por defecto devuelve las encontradas como Carlos cumple el WHERE, devuelve 1 aunque el valor ya fuera 500 y no hubiera cambio real. Si usara useAffectedRows=True daria 0.

(56) 12. Ana no tiene canjes. Se quiere listar a TODOS los estudiantes y, si existe, su canje de la recompensa 1. ¿Aparece Ana?

No aparece, hay que mover la condición al ON

```
SELECT e.nombre, h.id_canje
FROM estudiantes e
LEFT JOIN historial_canjes h
    ON h.id_estudiantes = e.id_estudiante
WHERE h.id_recompensa = 1;
```

Ana no tiene canjes por eso al realizar el LEFT JOIN, los campos de su historial queda en NULL, después la condición WHERE h.id_recompensa = 1 descarta esa fila porque NULL no cumple la condición.

(58) 13. ¿Qué devuelve request.getParameter("usuario") al enviar el formulario?

```
<form method="post" action="login">
<input type="text" id="usuario">
<button>Entrar</button>
</form>
```

Cuando se envía un formulario el navegador manda solo los campos que tienen el atributo name, usando ese nombre como clave, el atributo id sirve para css y javascript, no para enviar datos, al faltar name, el campo no viaja al servidor y getParameter("usuario") devuelve NULL.

(57) 14. ¿Qué ocurre al recorrer crs después de cerrar la conexión?

Funciona sin problema, cacheRowset guarda los datos en memoria

```
CachedRowSet crs = RowSetProvider
    .newFactory().createCachedRowSet();
try (Connection c = ...
     ResultSet rs = ...) {
  crs.populate(rs);
}
while (crs.next()) {...}
```

Un ResultSet normal necesita la conexión abierta porque va leyendo del servidor y un CacheRowset es desconectado: populate(rs) copia todas las filas al cerrar conexión ya tienes tu propia copia y puedes recorrerla con crs.next() sin error.

(56) 15

(55) 15. Se abre solado /hola sin enviar ningún parámetro nombre.
¿Qué muestra cada línea?

```
<p> <%= request.getParameter("nombre") %> </p>
<p> ${param.nombre} </p>
```

La línea 1 (<%= ... >) muestra la palabra NULL
La línea 2 (${param.nombre}) muestra vacío, porque EL oculta los nulos.
Ambas líneas buscan el mismo parámetro, pero lo procesan de manera diferente.

(53) 16. titulo = "Assassin's Creed"; ¿Qué se ve en el campo del formulario?

```
out.println("<input name='titulo' value='"
  + titulo + "'>");
```

Se ve solo Assassin, el apóstrofe de Assassin's" cierra el atributo value antes de tiempo y el HTML se rompe, además existe un riesgo de inyección HTML/XSS al insertar texto sin escapar

(54) 17. El usuario no inicio sesión, ¿Se ejecuta dao.eliminar(id)?

```
if (request.getParameter("usuario")
    == NULL) {
    response.sendRedirect("login");
    return;
}
dao.elimina(id);
```

Si se ejecuta dao.eliminar(id) porque falta un return, despues de SendRirect(), así que el codigo continua, además valida un parametro y no la sesión del usuario.

(57) 18. Canje: se descuentan los puntos y luego se registra el canje. El INSERT falla porque la recompensa no existe. Con autocommit por defecto, ¿Como queda la base de datos.

```
ps1 = C.prepareStatement("UPDATE estudiantes"
+ "SET puntos = puntos - 150"
+ "WHERE id-estudiantes = 1");
ps1.excuteUpdate();
ps2 = cprepareStatement("INSERT INTO"
+ "historial_canjes VALUES (NULL,1,99,NOW())");
ps2.excuteUpdate(); // Falla FK
```

Inconsistente, con autocommit el UPDATE ya quedo guardado así que el estudiante pierde 150 puntos pero no queda ningun canje, se arregla con SetAutoCommit(false); commit() y rollback() en el catch.

(56) 19. ¿Qué ocurre al ejecutar este CREATE TABLE?

```
CREATE TABLE canje2 (
  id INT PRIMARY KEY
  id_est INT NOT NULL
  FOREIGN KEY (id_est)
    REFERENCES estudiantes(id_estudiante)
    ON DELETE SET NULL
);
```

Se produce un error y la tabla no se crea porque ON DELETE SET NULL intenta asignar NULL a la columna id_est pero esta definida como NOT NULL.

(57) 20. nombre es VARCHAR(100) NOT NULL ¿Qué ocurre?

```
ps = conn.prepareStatement (
  "INSERT INTO estudiantes"
  + "(nombre, puntos) VALUES (?,?)");
ps.setString(1, null);
ps.setInt(2, 100);
ps.executeUpdate();
```

Se produce un error porque se intenta guardar un estudiante sin nombre, pero la base de datos exige que ese campo tenga un valor y no permite dejarlo vacío (NULL).

(57) 21. ¿Qué devuelve esta consulta

5 3 4 2

```
-- estudiantes: (1, 'Carlos Mendoza', 500),
-- (2, 'Ana Torres', NULL), (3, 'Luis Rojas', 200)
SELECT COUNT(*), COUNT(puntos)
FROM estudiantes;
```

COUNT(*) cuenta todas las filas, sin mirar su contenido! son 3 estudiantes

COUNT(puntos) cuenta valores no nulos, Ana tiene puntos NULL así que no se cuenta: quedan 2.

(57) 22. ¿Qué ocurre al ejecutar?

```
PS = conn.prepareStatement(
  "INSERT INTO estudiantes"
  + "(nombre, puntos) VALUES (?, ?)");
ps.setString (1, "Ana");
ps.executeUpdate();
```

Ocurre un error porque hay dos parámetros (?) en la consulta, pero solo se asigna el primero, falta asignar al segundo, correspondiente a puntos, antes de ejecutar.

57 23. ¿Cuántos juegos se insertan?

Solo 1 juego (x3)

```
String [] t = {"x1", "x2", "x3"};
for (String n:t) {
  ps.setString (1,n);
}
ps.addBatch ();
int [] r = ps.executeBatch();
```

setString solo prepara el valor de parametro, no guarda ninguna lista, quien agrega un orden de ejecución a la cola es addBatch (). en el código el for cambiaba el valor tres veces, pero el addBatch () se llamaba una sola vez al final, con el último valor (x3), por eso solo se inserta un juego, moviendolo dentro del ciclo se encolan las 3 inserciones.

(57) 24. puntos es INT y Carlos tiene 500. ¿Qué devuelve?

```
// SELECT puntos FROM estudiantes
//   WHERE id_estudiante = 1
rs.next();
String p = rs.getString ("puntos");
```

JBBC puede convertir automaticamente un INT a texto asi que getString("puntos") devuelve "500" sin error, pero conviene usar el método que corresponde al tipo getInt. para evitar conversiones innecesarias y asi podemos trabajar directamente con el número.

(58) 25. ¿Qué filas devuelve?

```
-- Luis Rojas
-- estudiantes : (1, 'Carlos Mendoza', 500),
-- (2, 'Ana Torres', NULL), (3, 'Luis Rojas', 200)
SELECT nombre FROM estudiantes
WHERE puntos <> 500;
```

Cualquier comparación con NULL (=, <>, >, <) da desconocido, que el WHERE trata como falso, entonces Ana nunca cumple puntos <> 500 y Carlos queda fuera porque 500 sí es igual a 500 solo Luis 200 pasa por el filtro.

(57) 26. Login hecho con concatenación. Si el usuario escribe exactamente `' OR '1'='1` ¿Qué ocurre?

```
String sql = "SELECT * FROM estudiantes"
  + "WHERE nombre = '" + usuario + "';
ResultSet rs = st.excuteQuery(sql);
boolean OK = rs.next();
```

El login puede aceptarse porque la condición '1'='1' siempre es verdadera, esto permite alterar la consulta y provocar una Inyección SQL, para evitarlo, se debe usar PreparedStatement.

(56) 27. ¿Con qué código HTTP llega esta respuesta?

```
protected void doPost (...) {
  if (!dao.existe(usuario)) {
  out.println("<p> Error : usuario "
    + " no encontrado </p>");
  }
}
```

Normalmente llega con el código 200 OK, aunque el texto diga Error, el servidor no sabe que fue un error, habría que usar response.sendError(...) o setStatus(...).

(56) 28. La columna puntos es INT DEFAULT 0 (admite NULL). ¿Qué valor de puntos tiene cada fila?

```
INSERT INTO estudiantes (nombre)
VALUES ('pepe'); -- puntos 0
INSERT INTO estudiantes (nombre, puntos)
VALUES ('Lolo', NULL); -- punto NULL
```

DEFAUL solo aplica cuando no mencionas la columna y si escribes NULL explícito, se queda en NULL.

Justus

(58) 29. Se abre /editar?id=(id vacío). ¿Qué responde Tomcat?

```
int id = Integer.parseInt(
  request.getParameter("id"));
```

HTTP 500 → Integer.parseInt("") lanza una NumberFormatException porque id está vacío, como la excepción no se captura, Tomcat responde con un error interno

(57) 30. Aparece: Access denied for user 'root'@'localhost' (using password: YES). ¿Qué significa?

```
DriveManager.getConnection(URL,
"root", PASSWORD);
```

Access denied significa que la conexión de red funciona, using password: YES indica que Java sí envía una contraseña.

La conexión llega al servidor MySQL, pero rechaza el acceso por credenciales incorrectas o porque el usuario root no tiene permisos para conectarse desde ese host, no es problema de puerto ni de URL.

(58) 31. historial_canjes tiene una FOREIGN KEY hacia estudiantes. ¿Qué pasa con esta instrucción?

```
TRUNCATE TABLE estudiantes;
```

Error, no se puede truncar una tabla referenciada por una FK; habría que borrar con DELETE porque se verifican las claves foráneas se borra primero la tabla hija (historial_canjes) y después la padre.

(57) 32. ¿Cómo salen ordenados los estudiantes?

```
PS=conn.prepareStatement(
  "SELECT nombre FROM estudiantes"
  + " ORDER BY ? DESC");
ps.setString(1, "puntos");
```

No se ordena por puntos un ? se manda como texto así que ordena por la cadena constante "puntos" y el orden es el natural, los nombres de columnas no se parametrizan.

(S6) 33. Precios: Among Us 15.50, Minecraft 59,90, Gran Turismo 7 189,00, FIFA 26 199,90, ¿Cuántos juegos devuelve?

```
SELECT titulo FROM juegos
WHERE precio BETWEEN 15.5 AND 189
```

BETWEEN incluye los dos extremos, por eso Among Us (15.50) y Gran Turismo 7 (189.00), Minecraft (59,90) sí cuentan. FIFA 26 cuesta 199.90 es mayor que 189 y se pasa.

(S6) 34. Según gamestore_db.sql, ¿Qué ocurre con este INSERT? Si ya existen estudiantes 1-2 y recompensa 1-2?

```
INSERT INTO historial_canjes
  (id_estudiante, id_recompensa)
VALUES (1,1);
-- fecha_canje TIMESTAMP
     DEFAUL CURRENT_TIMESTAMP
```

Se inserta bien id_canje se autogenera y fecha_canje toma la fecha y hora actual por el DEFAUL

(S7) 35. precio es DECIMAL(6,2) con 0.10 y 0.20. ¿Qué imprime 0.30000000000000004

```
double total = 0
while (rs.next()) {
   total += rs.getDouble("precio");
}
System.out.println(total);
```

0.30000000000000004, el tipo de double no representa 0.1 y 0.2 con exactitud, para dinero se puede usar BigDecimal con getBigDecimal().

(S8) 36. ¿Qué ocurre en la segunda línea? IllegalStateException: ya se uso getWriter()

```
response.getWriter().print("a");
response.getOutputStream();
```

El servlet puede escribir la respuesta como texto (getWriter) o como bytes (getOutputStream), pero solo una de los dos por respuesta, si ya llamaste a getWriter() y luego pides getOutputStream(), Tomcat lanza IllegalStateException.

(57) 37. Con Java 6+ y el .jar del conector en el classpath, es obligatorio llamar a Class.forName antes de getConnection()?

No, no es obligatorio

```
// class.forName(
    "com.mysql.cj.jdbc.Driver");
Connection c = DriverManager
  .getConnection(URL, USUARIO, PASSWORD);
```

Desde JDBC 4 (Java 6), el DriverManager descubre automáticamente los drivers que están en el classpath basta con tener el .jar del conector agregado al proyecto. Class.forName solo era necesario en versiones antiguas

(57) 38. La consulta no encuentra a nadie ¿Qué ocurre?

SQLException al leer no hay fila actual rs.next() devolvió false

```
rs = conn.prepareStatement(
  "SELECT nombre FROM estudiantes"
  + "WHERE 1 = 0").executeQuery();
rs.next();
String n = rs.getString(1);
```

rs.next() mueve el cursor a la siguiente fila y devuelve true si existe. Con cero resultados devuelve false y el cursor sigue antes de la primera fila, y leer la en ese estado lanza una SQLException, por eso siempre se revisa con if (rs.next()).

(57) 39. ¿Qué puntos queda guardado para Luis?

10 el parametro 2 conserva el valor anterior

```
ps = conn.prepareStatement("INSERT INTO "
  + "estudiantes (nombre, puntos)"
  + "VALUES (?, ?)");
ps.setString(1, "Ana"); ps.setInt(2, 10);
ps.executeUpdate();
ps.setString(1, "Luis");
ps.executeUpdate();
```

Los parámetros de un PreparedStatement se conservan entre ejecuciones hasta que los cambies, al reusar el mismo ps, el parámetro 2 sigue valiendo 10.

(58) 40. Dos alumnos abren /visitas al mismo tiempo desde PCs distintas. ¿Qué pasa con visitas?

```
@WebServlet ("/visitas")
public clas Visitas extends HttpServlet {
  private int visitas = 0;
  protected void doGet (...) {
    visitas ++;
    ...
  }
}
```

Tomcat crea una sola instancia del servlet y atiende a cada visitante en un hilo distinto, el campo visitas es compartido por todos. visitas ++ no es seguro ¡y se puede perder un conteo, usa: AtomicInteger o synchronized.

(57) 41. MySQL corre en el puerto 3306, pero la URL dice 3307. ¿Qué excepción aparece?

C.

```
DriveManager.getConnection (
  "jdbc:mysql://localhost:3307/"
  + "gamestore-db", "root", "clave");
```

CommunicationsException ("Communications link failure")
Nadie escucha en el puerto 3307 y la conexión es rechazada.

57 42. Con mysql-connector-j-9.7.0.jar (el de lib/), ¿Qué ocurre?
SQLException no se puede convertir el texto a número

```
// SELECT nombre FROM estudiantes
rs.next ();
int n = rs.getInt ("nombre");
// nombre = 'Carlos Mendoza'
```

getInt intenta convertir el contenido a número, Carlos Mendoza no es numérico así que driver lanza una SQLException de conversión. debe utilizar getString ("nombre").

(S6) 43. ¿Qué id recibe la nueva fila?

4

```
-- tabla con id AUTO_INCREMENT vacía
INSERT INTO ai (n) VALUES (1), (2), (3);
DELETE FROM ai WHERE id = 3;
INSERT INTO ai (n) VALUES (4);
```

AUTO_INCREMENT mantiene un contador que solo avanza nunca retrocede, después de asignar 1, 2 y 3 el siguiente número es 4, aunque borres el id 3

(S6) 44. Ana no tiene canjes ¿Qué número muestra la consulta para Ana?

1 (debería ser 0) hay que usar COUNT(h.id_canje)

```
SELECT e.nombre, COUNT(*) AS canjes
FROM estudiantes e.
LEFT JOIN historial_canjes h
  ON h.id_estudiante = e.id_estudiante
GROUP BY e.nombre;
```

El LEFT JOIN genera una fila para Ana, con las columnas de h en NULL. COUNT(*) cuenta filas y si esa fila existe, así queda 1.
COUNT(h.id_canje) cuenta los valores que no son NULL, así que para Ana da 0, que es lo correcto.

(S6) 45. Acción: Fortnite 0.00 y Free Fire 0.00; Deportes: FIFA 26 199.90 y Rocket League 0.00. ¿Cuántas filas devuelve?

3 filas

```
SELECT DISTINCT genero, precio
FROM juegos
WHERE genero IN('Accion', 'Deportes');
```

DISTINCT actua sobre la pareja (género & precio): Fortnite y free fire tienen la misma pareja (Accion, 0.00) así que cuentan como una. Deportes tienen parejas distintas.

S6 46. ¿Carlos (id 1) tiene canjes en historial_canjes, cuya FK se definió sin ON DELETE ¿Qué ocurre?

Error por clave foránea, Carlos no se borra

```
DELETE FROM estudiantes
WHERE id_estudiante = 1;
```

Error no se puede borrar el padre, sin ON DELETE, el comportamiento es RESTRICT y carlos no se borra

(55) 47. La aplicación está en el contexto /tienda y esta página se abrió desde /tienda/juegos/buscar.html. ¿A qué URL se envía cada formulario?

```
<form action="buscar">    <!-- A -->
<form action="/buscar">   <!-- B -->
```

A (buscar) va a /tienda/juegos/buscar, porque es una ruta relativa a la carpeta actual.
B (/buscar) va a /buscar, sin /tienda, y da 404

(56) 48. ¿Qué devuelve?
3.5000 y 3.

```
SELECT 7/2, 7 DIV 2;
```

En MySQL el operador / es división normal con decimales
DIV hace la división entera, descarta la parte decimal.

(57) 49 ¿Qué imprime?
el id del primer estudiante: "1".

```
Resultset rs = conn.prepareStatement(
  "SELECT id_estudiante, nombre"
  + "FROM estudiantes").executeQuery();
rs.next();
System.out.println(rs.getString(1));
```

getString(1) toma la primera columna, id_estudiante, no el nombre.

(56) 50. Con el modo estricto activo (el predeterminado en MySQL 8 y MariaDB), qué ocurre?
Error Data too long for column 'n'

```
CREATE TABLE corto (n VARCHAR(5));
INSERT INTO corto
VALUES ('Carlos Mendoza');
```

En modo estricto por defecto en MySQL 8 y MARIA DB rechaza los datos que no caben (INSERT) y sin modo estricto los cortaría en silencio, perdiendo información

(56) 51. Carlos tiene puntos = 500 y bono = 0 ¿Cuánto vale bono después del UPDATE?

1200

```
UPDATE estudiantes
SET puntos = puntos + 100,
    bono = puntos * 2
WHERE id_estudiante = 1;
```

MySQL aplica los SET de izquierda a derecha, así que bono usa puntos ya actualizado (600 x 2)

(57) 52. ¿Qué ocurre al ejecutar esto en Tomcat 11? Excepción (IllegalArgumentException) y HTTP 500

```
Cookie c = new Cookie ("usuario",
                       "Ana Torres") {
response.addCookie (c) }
```

Tomcat 11 no admite espacios sin comillas en el valor de una cookie. Hay que codificar el valor, por ejemplo con URLEncoder

(58) 53. Se hacen 3 peticiones GET al servlet ¿Cuántas veces se imprime "init" y cuantas "get"

"init" 1 vez y "get" 3 veces

```
public void init () {
  System.out.println ("init");
}
protected void doGet (...) {
  System.out.println ("get");
}
```

El servlet se crea una sola vez y cada petición llama a doGet.

(56) 54. Si la FK de historial_canjes se hubiera definido así, ¿qué pasa al borrar a Carlos (id 1)?

Se borra Carlos y automaticamente todas sus canjes

```
FOREIGN KEY (id_estudiante)
  REFERENCES estudiantes (id_estudiante)
  ON DELETE CASCADE
```

CASCADE propaga el borrado: al eliminar la fila padre, elimina solas las filas hijas que la referencian

(55) 55 ¿Qué muestra menu.jsp si el mensaje?

Aparece vacía

```
request.setAttribute("mensaje",
  "Bienvenido");
response.sendRedirect("menu.jsp");
```

sendRedirect genera una petición nueva y el atributo de request se pierde. Usa Forward mantiene la misma petición, así que el atributo sí llega a la JSP en un redirect guarda los datos en la sesión

(56) 56 ¿Qué campos llegan al servidor?

Llegan grado y juego

```
<input name="codigo" value="A1" disabled>
<input name="grado" value="5°" readonly>
<input name="juego" value="3"
  type="hidden">
```

Un campo disabled queda inactivo y el navegador no lo incluye al enviar el formulario, readonly solo impide editar, pero el campo se envía y hidden no se ve pero también se envía

(57) 57 Ana tiene puntos = NULL. ¿Qué imprime?

0 true

```
// SELECT puntos ... WHERE id = 2
rs.next();
int p = rs.getInt(i);
System.out.println(p + " "
  + rs.wasNull());
```

getInt devuelve 0 cuando NULL, y wasNull() te avisa que era NULL.

(58) 58 En torneo_gamestore_db los 10 juegos se reparten 2 por cada uno de 5 género. ¿Cuántas filas devuelve?

5 filas

```
SELECT genero, COUNT(*)
FROM juegos
GROUP BY genero
HAVING COUNT(*) >= 2;
```

GROUP BY agrupa por género, dando 5 grupos, HAVING filtra los grupos por su conteo. Cada grupo tiene 2 juegos, y 2 >= 2 es verdadero, así que los 5 grupos pasan.

(54) 59. Este método devuelve la lista de estudiantes. ¿Qué pasa cuando quien lo llama hace rs.next()?
SQLException: el ResultSet ya está cerrado

```
ResultSet listar() throws SQLException {
  try (Connection c = ConexionBD
      .obtenerConexion();
    PreparedStatement ps =
      c.prepareStatement(sql)) {
    return ps.executeQuery
  }
}
```

El try-with-resources cierra la conexión y el statement al hacer return, y arrastra al ResultSet, devuelve una List de objetos o un CachedRowSet.

(55) 60. El usuario NO marca la casilla. ¿Qué responde el servidor?
NullPointerException y HTTP 500

```
<input type="checkbox" name="acepto">
// Servlet
IF.(request.getParameter("acepto")
  .equals("on")) (...)
```

Un checkbox sin marcar no se envía, así que getParameter da null y null.equals(...) falla.
Escribes como "on".equals(...)

(56) 61. La tabla juegos usa utf8mb4_unicode_ci y tiene 'Fortnite' y 'Free Fire' con género = 'Accion'. ¿Qué devuelve?
Fornite y Free Fire

```
SELECT titulo FROM juegos
WHERE genero = 'accion';
```

El collation _ci ignora mayúsculas y también tildes así que que 'accion' coincide con 'Acción'.

(57) 62. Ordenados por precio de mayor a menor: Zelda 250, Mario Kart 8 249, FIFA 26 199,90, Gran Turismo 7, 189.
¿Qué devuelve
Mario Kart 8 y FIFA 26

```
SELECT titulo FROM juegos
ORDER BY precio DES
LIMIT 1, 2;
```

LIMIT 1,2 salta 1 fila (zelda) y toma las dos siguientes

(58) 63. ¿Compila este servlet?

No compila

```
protected void doGet (HttpServletRequest q,
    HttpServletResponse r)
    Throws ServletException, IOException {
  Connection c = DriverManager
    .getConnection (URL, U, P);
```

getConnection lanza SQLException, que es checked, y el método solo declara ServletException e IOException.

(57) 64 ¿Qué ocurre en la última línea?

SQLException en rl.next()

```
Statement st = conn.createStatement();
ResultSet rl = st.executeQuery (
  "SELECT nombre FROM estudiantes");
ResultSet r2 = st.executeQuery (
  "SELECT titulo FROM juegos");
rl.next ();
```

Un statement solo puede tener un ResultSet abierto, así que la segunda así que la segunda consulta cerro rl. Usa dos statement.

(55) 65. El servlet /login usa una de estos dos. ¿Qué diferencia ve el usuario en la barra de direcciones?

```
// option 1
request.getRequestDispatcher ("menu.jsp")
    .forward (request, response);
// option 2
response.sendRedirect ("menu.jsp");
```

Con forward la barra sigue en /login; con redirect cambia a menu.jsp

Forward ocurre dentro del servidor, el navegador no se entera del cambio por eso el URL no cambia y los atributos del request se conservan.
sendRedirect responde con una orden de navegador (ve a menú.jsp), este hace una petición nueva y la barra muestra la nueva dirección

(58) 66. ¿Qué ocurre en sendRedirect?

IllegalStateException

```
PrintWriter out = response.getWriter();
out.println("<h1>Hola</h1>");
out.flush();
response.sendRedirect("ranking");
```

El flush() ya envió la respuesta committed, y después no se puede redirigir, el navegador solo ve "Hola

(58) 67 ¿Qué devuelve s si el visitante nunca inició sesión?

null

```
HttpSession s = request.getSession(false);
```

Con getSession(false) no se crea sesión si no existía

(56) 68. ¿En qué orden salen los nombres?

Ana, Luis, Carlos

```
-- estudiantes: (1, 'Carlos Mendoza', 500),
-- 2, 'Ana Torres', NULL), (3, 'Luis Rojas', 200)
SELECT nombre, puntos FROM estudiantes
ORDER BY puntos ASC;
```

Ana (NULL), Luis (200), Carlos (500) en MySQL, NULL va primero en orden ascendente.

(58) 69 Se marcan AMBAS casillas y se envía. ¿Qué contiene g?

Acción

```
<input type="checkbox" name="genero"
       value="Accion">
<input type="checkbox" name="genero"
       value="Deportes">
// Servlet:
String g = request.getParameter("genero");
```

Cuando varios campos comparten el mismo name, el navegador envía todos los valores. getParameter solo devuelve el primero
getParameterValues devuelve un arreglo con todos los marcados

(55) 70. Al desplegar en Tomcat 11, ¿qué ocurre con este mapeo?

La aplicación no despliega

```
@WebServlet("login")
public class LoginServlet
    extends HttpServlet {...}
```

Falla con IllegalArgumentException por un url-pattern inválido, debe ser "/login, con barra inicial

(56) 71. historial_canjes tiene una fila con id_estudiante = NULL. ¿Qué devuelve la consulta para "alumnos que nunca canjearon"?

Ninguna fila

```
SELECT nombre FROM estudiantes
WHERE id_estudiante NOT IN
  (SELECT id_estudiante
   FROM historial_canjes);
```

NOT IN con un NULL en la lista nunca es verdadero por eso solo aparece una sola fila NULL y NOT IN deja de devolver resultados, usaríamos NOT EXISTS o agregar WHERE id_estudiantes IS NOT NULL al subselect

(57) 72. ¿Qué ocurre en getGeneratedKeys()?

SqlException: "Generated keys not requested

```
ps = conn.prepareStatement(
"INSERT INTO estudiantes (nombre)"
+" VALUES ('Zoe')");
ps.executeUpdate();
ResultSet K = ps.getGeneratedKeys();
```

Faltó prepareStatement(sql, statement.RETURN_GENERATED_KEYS)

58 73. Un compañero arma un login con este formulario. ¿Cuál es el problema principal?

Usa Get

```
<form action="login" method="get">
  <input name="usuario">
  <input name="clave" type="password">
</form>
```

la clave viaja en la URL, visible en la barra, el historial y los logs. debe ser method="post"

(56) 74. MiRankingWeb registra el servlet en "/ranking".
¿Qué responde Tomcat a http://localhost:8080/ranking (con barra al final)?

404 Not Found.

```
contexto.addServletMappingDecoded(
"/ranking", "RankingServlet");
```

El mapeo /ranking es exacto y no incluye la barra final.

(56) 75. Recompensas tiene 2 filas (costo 150 y 300). ¿Qué ocurre?

Error 1242: Subquery returns more than 1 row

```
SELECT nombre FROM estudiantes
WHERE puntos =
  (SELECT costo_puntos
   FROM recompensas);
```

El operador = espera un único valor, pero la subconsulta devuelve dos (150 y 300), lo que provoca error.
IN compara contra una lista completa y resuelve el problema.

(57) 76. ¿Compila este código (Java 9 o superior)?

```
Connection conn = ConexionBD
    .obtenerConexion();
try (conn) {
    // usar conn
}
```

Sí compila en Java 9 o superior porque conn es "efectivamente final" y puede ir dentro de try (conn). Eso sí, falta atrapar o declarar SQLException por el close().

(57) 77. Un alumno quiere reutilizar el DAO para varias tablas. ¿Qué ocurre?

Error de Sintaxis SQL

```
ps = conn.prepareStatement(
  "SELECT * FROM ?");
ps.setString(1, "estudiantes");
rs = ps.executeQuery();
```

El ? se manda como 'estudiantes' entre comillas, y eso no vale como nombre de tabla, los nombres de tablas y columnas no se parametrizan; valida contra una lista permitida.

(67) 78. En Linux se creó la base de datos gamestore_db (en minúsculas). ¿Qué ocurre?

Error. Unknown database 'GameStore_db'

```
DriverManager.getConnection(
  "jdbc:mysql://localhost:3306/"
+ "GamStore_db", "root", "clave");
```

En Linux los nombres de base de datos distinguen mayúsculas y minúsculas

(67) 79. La tabla tiene 3 estudiantes. ¿Qué imprime este código?

Solo Carlos Mendoza

```
ResultSet rs = ps.executeQuery();
if (rs.next()) {
  System.out.println(
  rs.getString("nombre"));
```

IF ejecuta su cuerpo una sola vez, así que lee solo la primera fila

While repite mientras next() devuelve TRUE y recorre todos las filas

(56) 80. ¿Qué promedio devuelve?

350

```
-- estudiantes: (1, 'Carlos Mendoza', 500),
--              (2, 'Ana Torres', NULL), (3, 'Luis Rojas', 200)
SELECT AVG(puntos) FROM estudiantes;
```

AVG ignora los NULL, así que calcula (500+200)/2 y no entre 3.